

新手教程-新详情页页面框架和结构

零、设计思想

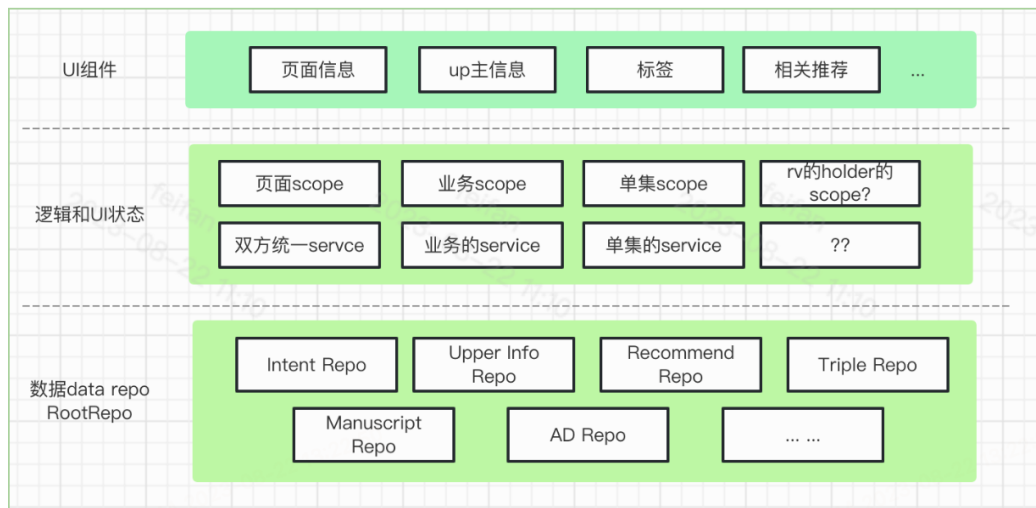
- 视图层(View)、数据层 (Data、Store) 、逻辑层 (Action、Reducer) 分离
- 单一数据源，单向数据流
- 状态驱动，状态的变化可预测

一、名词解释

- UI、UI Component、UI组件，统一在下文叫做UI。被严格定义为一个读取状态进行UI绑定的组件，它只做状态到UI的绑定。
- UI State，统一在下文叫做State。被严格定义为UI的状态，描述当前UI应该显示什么内容、长什么样子等。
- PageService、BusinessService，统一在下文叫做Service。被定义为逻辑处理模块以及对应UI State的发射模块。
- Repository，统一在下文叫做Repo，存放接口数据、不同交互数据的存储器。下文会详细解释。
- Scope、作用域，统一在下文被叫做Scope。有两重含义，Dagger2依赖注入提供的对应作用域，以及此Dagger2作用域下运行着的协程作用域。
- Component，Dagger2内提供的一个单独的模块。可以包含许多的业务处理的Service和Repository。
- Module，Dagger2内提供的可以被安装到不同Component的依赖注入对象的提供工厂。
- Driver，驱动对应Dagger Scope的对象，有UGC的someDriver、OGV的someDriver。
- View接口请求、PlayView接口请求，下文统一成做View和PlayView，对应的是详情页的页面接口请求和播放接口请求。

二、详细设计

2.1 基础概念及分层



- UI Component

UI为一个个单一组件，它只关注它自身的State。它会持有service的依赖。它在不停地观察service中的state的变化，以达到UI响应State的目的。同时UI需要进行逻辑操作时，是在调用service的函数进行操作，后续进行的逻辑变化最后都会变成state让ui进行相应操作。

- **Scope**

scope是一个作用域，它代表了这个作用域的生命周期，在页面内，我们定义它有三个作用域。同时对应的也是coroutine scope，协程的scope也会同步的跟随对应的生命周期。

- **Service**

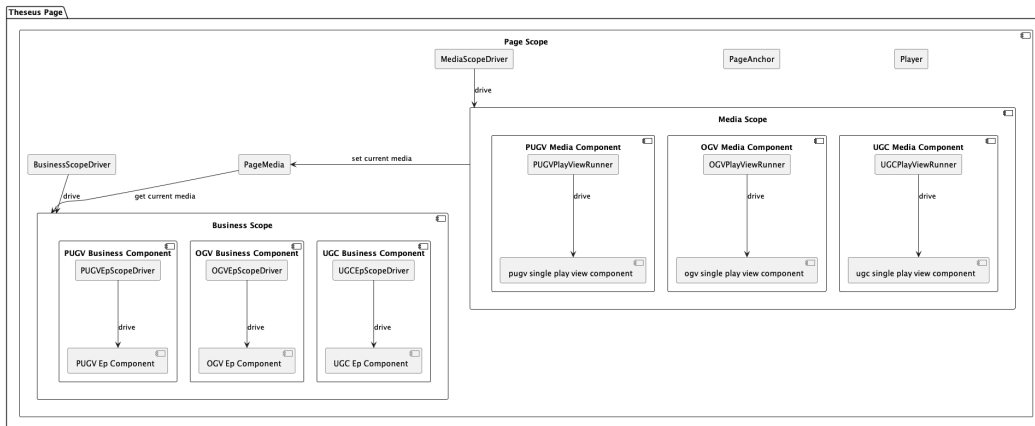
service作为负责处理逻辑和进行state输出的模块，它可以被称作为一个胶水层。一个service本质上对应的是一个业务和功能模块，它应该和UI进行一个一一对应的关系（推荐）。同时service会限定scope，它只允许被运行在指定的scope下，它不能超过它的scope去访问别的内容。同时service也不允许互相依赖和持有，但是service被允许持有多个repo，通过来自不同repo的数据，进行一个业务逻辑和state的处理。

service的创建和生命周期依赖scope，会统一使用依赖注入框架来进行管理。

- **Data Repository**

repo作为接口请求数据处理的模块，它是一个很小颗粒度的数据存储和请求模块。一个repo内对应的可能只是一个业务模块下的一小块的数据，它负责处理和存储这份数据。当它被更新的时候，需要把这份数据通过flow更新出去，每个持有和订阅这份数据的人，都可以响应对应数据的更新。repo之间应该尽量避免互相依赖。

2.2 Scope详细介绍



上图是一个简易的用PlantUML大致画得结构图。我们从上往下看，大致分为PageScope、MediaScope、SinglePlayViewScope、BusinessScope、EpisodeScope五种。下面我们从页面开始，逐级往下讲述。

- **PageScope**

很简单的理解，就是一个详情页的页面是一个单独的作用域，每启动一个新的详情页，会创建一个新的PageServiceComponent。同时也很自然的，绑定上了页面的CoroutineScope，也就是Activity的lifecycle scope。

具体PageServiceComponent的创建依赖以及对应的Install Module，大家可以自行去看com.bilibili.ship.theseus.all.di.PageServiceComponent的代码。

PageServiceComponent创建起来之后，自然页面上的各个Service和Repo也都开始运转，MediaScope和BusinessScope的Driver也开始执行工作了。

- **MediaScope**

简单来说就是一次播放的作用域，负责处理播放相关的行为和业务。每发起一次播放，都会创建一个MediaComponent，分为各自业务的MediaComponent。每一次的MediaComponent由MediaScopeDriver内的一个StateFlow驱动。注意，这里的每一次MediaScope都对应着一次播放，不管是切一个新的视频，切一个视频的一P，一季视频的一集。甚至包括小窗和story模式下共享播放回来重新开始播放，都意味着一次全新的MediaComponent的创建。

- **SinglePlayViewScope**

我们的播放接口被称作PlayView，在每次播放中，可能会触发多次play view的接口请求。因此很明确的。我们在每次play view获取到准确的数据之后，都会驱动起一个SinglePlayViewScope。

- **BusinessScope**

在融合详情页，不同的业务自然会有不同的详情页，因此针对不同的业务，我们设计了独有BusinessScope。对应的其实是View接口的请求。也就是一次详情页业务的Component。例如我们切换了一个视频，需要重新请求一次View接口，那么我们会切换一次BusinessScope，不管它是切换到UGC/OGV/PUGV。每一次的切换都由BusinessScopeDriver内的StateFlow来驱动。当BusinessScope切换了，我们也自然从一个业务页面切换到了另一个业务页面。

- **EpisodeScope**

在详情页内，我们有许多页面的数据会从view接口下发，例如UGC的合集，多P。OGV的季度信息啥的。我们需要依赖这些数据处理很多业务，同时处理这些业务的过程中，我们还需要关注当前正在播放的内容。因此设计了EpisodeScope，它会在BusinessScope内。等到MediaScope运转起来，获取当前的Media数据，然后组合出一个新的状态，并且用这个状态驱动起EpisodeScope。

2.3 Scope设计目的

上面简单介绍了一下我们的Scope的定义，那么大家必然会有一些疑问，就是我们为什么设计这么多的作用域？我该怎么使用这些作用域？我该怎么定义一些业务在什么作用域内？在不同的作用域内我们能做什么事？下面会针对这些问题做一一的解答。

我们为什么设计这些scope

新同学可能没有什么概念，详情页作为bilibili最主要的播放场景之一，承载了大量的业务，代码数量级也是极其恐怖的。在众多的业务中，复杂的业务关系，复杂的业务逻辑是需要大家都去面对的。而详情页内，不仅有普通的页面业务，还有复杂的和用户交互的播放业务，还包括了和弹幕的交互，投屏的交互。而且，这样的详情页在我们的APP内还不止一个。还有包括OGV/PUGV在内也都有自己的详情页。为了回答设计Scope问题，我们先要提出一些问题。如何把复杂的UGC、OGV、PUGV业务融合到一起，我们采取了什么方案呢？答案就是Dagger2+协程。

为什么用Dagger2+协程

Dagger2是一个依赖注入工具，而依赖注入是IOC控制反转的一种思路。控制反转看起来很高大上，其实很简单，你可以认为它只做了一件事，就是在写程序时，例如对象A使用了对象B的功能，那我们一般会A去创建B。但是其实A只是需要B的一些开放的函数，并不需要创建B。因此我们通过把这个流程放到框架中来实现。利用Dagger2，我们编写A，声明对B的依赖，那么A在实例化的时候，自动会把B给传入，不需要我们主动地去创建B，从而达到A和B解除创建关系的逻辑。可能大家会觉得，好像也就做了一件很简单的事。没有很多的能力，我自己创建B好像也没啥问题啊。但是大家可以想象一下我们的详情页，有茫茫多的场景需要去使用播放器，调度播放器，监听播放器的各种事件和状态，那么所有的组件都去创建一个播放器？这不可能，那么

这个播放器放在什么地方，每个逻辑和业务如何去获取这个播放器？这里就是依赖注入的价值体现了。

同时在dagger2里，它提供了Scope和Component，可以清晰地划分出模块的依赖以及能够声明的依赖关系。这块我就不展开了。简单来说就是父子关系下，子作用域内可以取到父作用域内的所有东西，**越小的作用域内确定的信息会更多，这是很重要的一点**，后面会展开讲解。

使用协程原因也很简单，简单来说就是协作式的执行代码。通过协程，我们能够去把状态事件等很明确的拆分开，并且通过协程的基础能力，例如结构化的并发，我们可以很好的解决内存泄漏的问题。这个后面也会展开说明。并且协程是非常好的单项数据流和基于数据流编程的工具。同时因为协程+kotlin带来的一些函数式的编程思想，是非常好的编程工具，利用这样的工具。你是能够写出更健壮和更安全的代码的。

利用协程提供的StateFlow，我们使用状态来描述当前的Component，当前运行的每一个Component，都是一个当前运行中的状态，每一次的StateFlow的状态切换，都意为着一个Component的切换创建和销毁。

(ps：因为这次的主题是详情页，这块就不展开说明了。后续如果大家感兴趣的话，可以再拉个分享，抛砖引玉。聊聊kotlin的协程、函数式编程啥的)

ok，现在我们可以回到第一个问题了，我们为什么设计这些Scope。

Dagger2中的Scope和Component，可以严格定义在不同的Scope下，被确定的信息，以及被使用的模块。同一Scope下的不同业务的Component，可以尽可能的复用类似或者相同的Module、Service、Repo等。这样的话，能够尽可能的统一业务和逻辑，并且也提高了代码的复用率。

从最顶层的PageScope说起，在PageScope内，自然是页面级别的状态逻辑业务，例如页面的全屏半屏状态、页面唯一的播放器、页面的性能买点收集器、状态栏管理等等等等。自然地，我们也给这边的PageScope挂上了对应的CoroutineScope，也就是页面Activity的LifecycleCoroutineScope，所有在PageScope内的Component等业务代码，使用对应的PageCoroutineScope，都会在页面的CoroutineScope cancel的时候自动进入cancel态，不会出现任何的泄漏。

页面的Scope讲完了，我们来讲讲具体业务的Scope

- 业务的Scope

大家看到页面，可以很明显的看到上半部分的播放区域和下半部分的简介区域。自然地，上半部分是播放，下半部分是简介。那么在这里，针对播放，我们设计了MediaScope和SinglePlayViewScope。在进入一次播放的时候，我们会根据播放的参数，构建起一个MediaScope和内部的Component，同时给当前的player绑定上一个playable，并且在这个MediaScope运转内，这个playable是不会发生变更和变换的。自然地，离开这次播放，就会cancel掉这次的MediaScope，也包括将player内的playable置空，同时CoroutineScope也会被cancel掉。

同时在播放的时候，每一次的PlayView请求，都会产生一次结果，那么我们根据这个结果，可以构建出SinglePlayViewScope，它活跃在MediaScope下，一次MediaScope可能会产生运转多次SinglePlayViewScope。这里我们可以用到👉的一句话，越小的作用域内确定的信息会更多，这里，我们的PlayView被完整的确定下来了，同时在SinglePlayViewScope内，我们也可以获取的到MediaScope下的所有信息。那么自然的，在SinglePlayViewScope内编写代码，我们需要判断的条件是更少的，我们的心智负担是更低的。大家编写业务代码也会更轻松便捷。

自然地BusinessScope就是简介下的作用域，它会由View来驱动。在BusinessScope下，业务类型，View接口的数据自然也是确定的。这里大家可能会产生新的疑问，EpisodeScope是啥。它其实是由MediaScope产生的一次播放和BusinessScope下的数据组合出的一个Scope。例如UGC有多P，OGV有一季下的一集。那么这个一P或者一集内，其实对应的就是EpisodeScope，在这个作用域内，整个详情页的业务逻辑数据基本上可以达到最小的颗粒度。在触发了切集切P后，自然也会调度到新的EpisodeScope。还是👉的那句话，越小的作用域内确定的信息越多，大家编写代码的心智负担就越低。这样出问题的可能性会更小。

ok，到这里，我们算是解答了我们为什么设计这么多的作用域。

2.4 如何使用Scope

接下来看后面的问题。我们可以通过一个🔴来看看如何解答我该怎么使用这些作用域？我该怎么定义一些业务在什么作用域内？在不同的作用域内我们能做什么事？

我们可以看看播放器起播的时候需要显示的toast和告诉播放器起播进度的需求。

我们知道用户有观看历史记录，历史记录可能来自于路由参数、服务端历史记录、用户本地历史记录。这个记录决定了我们起播之后让播放的playable seek到那个位置。然后是toast，这个提示的文案需要等待Play View接口。那么很自然的。我们可以把这个业务写在MediaScope里，伪代码如下。

```
1 @MediaScope
2 class StartPositionService @Inject constructor(
3     @MediaCoroutineScope scope: CoroutineScope,
4     playable: TheseusPlayable,
5     startPos: Duration?,
6 ) {
7     init {
8         scope.launch(){
9             playable.seekTo(startPos)
10        }
11    }
12 }
13
14 @SinglePlayViewScope
15 class StartToastService @Inject constructor(
16     @SinglePlayViewCoroutineScope scope: CoroutineScope,
17     playable: TheseusPlayable,
18     playViewReply: PlayViewReply
19 ) {
20     init {
21         scope.launch{
22             val text = playViewReply.getToast()
23             showPlayerToast(text)
24         }
25     }
26 }
```

ok简单来说，构建起了这些作用域，我们在开发需求的时候首先要思考的是，这个需求，在什么作用域下？它需要了哪些数据，它是如何工作的，尽可能的在更小的作用域内去思考问题，还是那句话，小的作用域内，你确定的信息更多，能够写出的代码更加的完善，需要处理的状态和判断逻辑会尽可能的少。针对这样，我们能够更简单和轻松地编写出不容易出现bug和各种问题的业务代码。

2.5、UI Component、State、Service、Data Repository

我们在业务中大概率是需要写UI，写状态，写各种逻辑和条件判断的。根据不同的条件去展示不同的UI和用户进行不同的交互，这是APP最主要的职责。因此我们抽象了UIComponent这个概念。

```
1 interface UIComponent<E : UIComponent.ViewEntry> {
2     fun createViewEntry(@UiContext context: Context, parent: ViewGroup? = null)
```

```

3
4 suspend fun bindToView(viewEntry: E)
5
6 fun viewReusingKey(): Any = this.javaClass
7
8 fun contentEqualityKey(): Any = this
9
10 fun identityEqualityKey(): Any = contentEqualityKey()
11
12 interface ViewEntry {
13     val root: View
14
15     class ViewViewEntry<V : View>(val value: V) : ViewEntry {
16         override val root: View
17             get() = value
18     }
19
20     class ComposeViewEntry(val value: ComposeView) : ViewEntry {
21         override val root: View
22             get() = value
23     }
24
25     class ViewBindingViewEntry<V : ViewBinding>(val value: V) : ViewEntry {
26         override val root: View
27             get() = value.root
28     }
29 }
30 }

```

UIComponent的设计很简单，它包含了一个创建对应ViewEntry的函数，以及一个suspend的bindToView的函数。

为什么这么设计？因为我们希望UIComponent是一个基于状态来描述UI的组件，那么为什么这么定义，我们在编写代码的时候如何理解并写出正确的UIComponent呢？

我们来看一个示例。

```

1 class ViewBindingUIComponentSample(private val state: StateFlow<TextState>)
2     UIComponent<UIComponent.ViewEntry.ViewBindingViewEntry<TheseusViewBindingComponentSample>>() {
3         override fun createViewEntry(
4             context: Context,
5             parent: ViewGroup?,
6         ): UIComponent.ViewEntry.ViewBindingViewEntry<TheseusViewBindingComponentSample> {
7             val binding = TheseusViewBindingComponentSampleBinding.inflate(layoutInflater, parent, true)
8             return UIComponent.ViewEntry.ViewBindingViewEntry(binding)
9         }
10
11         override suspend fun bindToView(
12             viewEntry: UIComponent.ViewEntry.ViewBindingViewEntry<TheseusViewBindingComponentSample>,
13         ) {
14             state.collectLatest { state -> //在挂起函数内把UI状态绑定到对应的view entry
15                 viewEntry.value.theseusTextSample.text = state.text
16             }
17         }
18     }

```

```

18
19     data class TextState(val text: String)
20
21 }
22
23 class ViewBindingUIService {
24     private val _flow = MutableStateFlow(ViewBindingUIComponentSample.TextSt
25     private val component = ViewBindingUIComponentSample(_flow.asStateFlow()
26
27     fun show(parent: ViewGroup) {
28         MainScope().launch {
29             val entry = component.createViewEntry(parent.context)
30             parent.addView(entry.root)
31             component.bindToView(entry)
32         }
33     }
34
35     fun someLogic() {
36         MainScope().launch {
37             delay(100) // 模拟耗时操作
38             _flow.value = ViewBindingUIComponentSample.TextState("标题2")
39         }
40     }
41 }

```

我们可以看到，我们定义了一个ViewBinding类型的UIComponent，它的构造函数是一个stateFlow。它提供了创建出一个view entry的函数，可以看到UIService在show的时候，通过Component创建了一个ViewEntry，并且把它添加到了view树上，然后让触发UIComponent的bindToView操作。UIComponent会开始收集状态流内的状态，并且把状态映射到UI上。如果在Service内触发了某些逻辑，我们将会更改状态，只要Component还在scope内持续bindToView，那么UI State将不断更新到View上。

这里很重要的一点就是，我们在用一个状态来描述UI，而UIComponent就是一个基于状态来针对UI做绑定的组件，它并不是一个UI。怎么理解呢？我们现在的Component内装载的是一个Android的View，如果是Compose函数，那么它依然可以保持工作，只需要将这个View替换为对应的Compose函数即可。如下

```

1 class ComposeUIComponentSample(private val state: StateFlow<ViewBindingUICon
2     UIComponent<UIComponent.ViewEntry.ComposeViewEntry> {
3     override fun createViewEntry(context: Context, parent: ViewGroup?): UICo
4         val composeView = ComposeView(context)
5         return UIComponent.ViewEntry.ComposeViewEntry(composeView)
6     }
7
8     override suspend fun bindToView(viewEntry: UIComponent.ViewEntry.Compose
9         viewEntry.value.setContent {
10             BiliTheme {
11                 val state = state.collectAsState()
12                 Title(state.value.text)
13             }
14         }
15     }
16

```

```
17     @Composable
18     private fun Title(title: String) {
19         ClickableText(text = AnnotatedString(title), modifier = Modifier.wid
20     }
21 }
```

在这种情况下，因为Compose是声明式和函数式的，那么我只要有状态，我就一定能描述清楚UI。这也是为什么UIComponent设计成这样的原因，它是基于状态来设计的。这里遵循了一个非常重要的概念，状态向下，事件向上。通过状态来描述一个UI，而不是拿着一个UI组件在不停地改变它所持有的状态。（ps:状态向下事件向上我个人觉得是Compose内很重要的一点，以及这边和函数式编程也有关系，后续可以再聊）

Service可以理解为ViewModel或者别的啥，它主要就是负责处理业务逻辑和构建状态，然后发射给对应的UIComponent。Repository则是负责进行Service之间的通信，以及存储远端数据等等。这里要说明的是，尽可能的遵循，Service之间不互相依赖，如果有需求需要通信，尽可能通过Repository进行。如果业务的Service没有和别的地方交互的需求，那么甚至它可以不存在Repository，只需要Service进行业务逻辑即可。（TODO，示例待完善）